

Stage 3 实验报告

2022011223 熊泽恩

实验内容

Step 6

为了完成这次实验，我进行了以下工作：

首先，我添加了 `scopestack.py`，完成了 `ScopeStack` 类，实现了**作用域栈**的功能。这里，我设计了两个成员函数：

- 新建了 `newScope` 函数，实现新建一个（局部）作用域，并将其压入作用域栈内。
- 新建了 `lookup` 函数，实现了按照“从栈顶到栈底”，即从内层作用域到外层作用域的顺序，查找能否获取符号。设计这个函数，其实是为了能够复用原来 `Scope` 的代码，这样我只需要把 `Scope` 这一类型替换为 `ScopeStack`，而不用修改它的成员函数调用相关的语句。

然后，我按照实验文档，修改了 `typer.py`，将 `Visitor[Scope, None]` 改为了 `Visitor[ScopeStack, None]`。同时，我把原来所有涉及到 `Scope` 的函数参数，全部修改为 `ScopeStack`。

最后，我修改了 `namer.py` 中的相关语句，把 `ctx` 的类型改为 `ScopeStack`。同时，

- 修改了 `Namer.visitFunction` 和 `Namer.visitBlock`，在函数开始时局部作用域**压栈**，并在函数结束时把对应作用域**退栈**。
- 修改了 `Namer.visitDeclaration`，只在当前的**栈顶作用域**里查找符号是否被声明。

Stage 3 思考题

Step 6

第 1 题

请画出下面 MiniDecaf 代码的控制流图。

```
1  int main() {
2      int a = 2;
3      if (a < 3) {
4          {
5              int a = 3;
6              return a;
7          }
8      return a;
9  }
10 }
```

上述代码可以写成以下中间代码：

```

1  FUNCTION<main>:
2      _T1 = 2
3      _T0 = _T1
4      _T2 = 3
5      _T3 = (_T0 < _T2)
6      if (_T3 == 0) branch _L1
7      _T5 = 3
8      _T4 = _T5
9      return _T4
10     return _T0
11 _L1:
12     return

```

可以被分为以下 3 个基本块：

B1:

```

1  FUNCTION<main>:
2      _T1 = 2
3      _T0 = _T1
4      _T2 = 3
5      _T3 = (_T0 < _T2)
6      if (_T3 == 0) branch _L1

```

B2:

```

1      _T5 = 3
2      _T4 = _T5
3      return _T4

```

B3:

```

1      return _T0

```

B4:

```

1 _L1:
2     return

```

流程图为：



